

2) Bastion Repository Setup (Air-gapped Environment)

Perform the offline installation on the **Bastion server** within the air-gapped (closed network) environment. This server will act as a repository, providing files and images to the Kubernetes (K8s) nodes.

2.1. Extracting the Offline Bundle

Extract the `outputs.tar.gz` file transferred from the external network.

```
# Extract the file at the location where 'outputs.tar.gz' is located.  
$ tar xzvf outputs.tar.gz
```

```
# Navigate to the generated 'outputs' directory.  
$ cd outputs
```

2.2. Modifying Local Repository (Nginx) Configuration

Modify the Nginx configuration included in the `outputs` bundle to ensure K8s nodes can correctly locate YUM packages (`rpms`) and binaries (`files`).

Issue

When using the original file, a 404 Not Found error occurs when accessing repository paths like `/files` or `/rpms/local`.

Cause

The original configuration placed the root `/usr/share/nginx/html;` directive inside the location `/` block. Since Nginx does not share settings between location blocks, the `/files` path could not locate the root path.

Solution

Moved the root directive outside the location `/` block to the server block as a common setting.

```
# Open the 'nginx-default.conf' file.  
$ vim nginx-default.conf
```

Original:

Nginx

```

server {
    listen 80;
    listen [::]:80;
    server_name localhost;
    location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
    }
    error_page 500 502 503 504 /50x.html;
    location = /50x.html {
        root /usr/share/nginx/html;
    }
    # Force sendfile to off
    sendfile off;
}

```

Overwrite the entire content of the file with the configuration below.

Modified Version:

Nginx

```

server {
    listen 80;
    server_name localhost;

    # Root directory inside the Nginx container
    # (setup-all.sh mounts the 'outputs' directory to this path.)
    root /usr/share/nginx/html;

    # --- /files (Binaries) Path Configuration ---
    location /files {
        autoindex on; # Allow directory listing
        sendfile off;
    }

    # --- /rpms/local (YUM) Path Configuration ---
    location /rpms/local {
        autoindex on; # Allow directory listing
        sendfile off;
    }

    # --- Root / Path ---
    location / {
        # Display a simple message instead of 403 Forbidden when accessing root
        return 200 "Kubespray Offline Repository Server is running.";
    }

    error_page 500 502 503 504 /50x.html;
    location = /50x.html {
        root /usr/share/nginx/html;
    }
}

```

2.3. Running Offline Services

Execute the `setup-all.sh` script to build the offline repository environment on the Bastion server.

```
# This script performs the following tasks sequentially:
# 1. (Nginx) Runs the File/RPM server as a container using the modified 'nginx-default.conf'.
# 2. (Registry) Runs the private container image registry.
# 3. (Image Load) References 'images.list' to load (push) all .tar.gz images
#    from the 'images/' directory into the private registry.
$ ./setup-all.sh
```

`setup-all.sh` Sub-script Execution Details:

- `run ./setup-container.sh` **(Prepare Container Environment)**
 - Installs the container runtime (containerd/nerdctl) on the Bastion server.
 - Pre-loads (caches) only the core service images (nginx, registry) locally to prepare for launching Nginx and Registry containers in the next steps.
- `run ./start-nginx.sh` **(Start Web Server)**
 - Runs the container using the pre-loaded nginx image.
 - This Nginx server acts as a **File Server** (binaries, Helm charts, etc.) and a **YUM Repository** (.rpm packages) that K8s nodes will access.
- `run ./setup-offline.sh` **(Local YUM Setup)**
 - Generates and configures the `offline.repo` file so that the Bastion server's own YUM/DNF settings point to the local Nginx server (`http://localhost/rpms/...`) instead of the internet.
- `run ./setup-py.sh` **(Local PyPI Setup)**
 - Configures `pip` to point to the local Nginx server (`http://localhost/pypi/`) instead of the internet.
 - (This step must complete successfully for `pip install -r requirements.txt` to work in the air-gapped network.)
- `run ./start-registry.sh` **(Start Private Registry)**
 - Uses the pre-loaded registry image to run a private image registry container (e.g., `172.30.31.14:35000`) that will store all K8s container images.
- `run ./load-push-all-images.sh` **(Image PUSH)**

- Uses `nerdctl` to load all K8s cluster images (.tar.gz files) from the `outputs/images/` directory and pushes them to the local private registry just started.

2.4. Setting Up Python Virtual Environment

Create an isolated Python environment for running Kubespray (Ansible).

```
# Kubespray 2.29.0 recommends Python 3.11 or higher.  
$ python3.11 -m venv ~/.venv/3.11  
  
# Activate the generated virtual environment. (Terminal prompt changes to '(3.11)' etc.)  
$ source ~/.venv/3.11/bin/activate  
  
# Check the activated Python version.  
(3.11) $ python --version
```

2.5. Extracting Kubespray Playbook

Extract the Kubespray (Ansible Playbook) included in the download bundle.

```
# The 'kubespray-2.29.0' (or configured version) directory will be created.  
(3.11) $ ./extract-kubespray.sh
```